

부산정보영재교육원

연구 보고서

이산수학 개념을 활용한
안전한 메시지·파일 전송 시스템

소 속	부산정보영재교육원 (중3 프로그래밍응용1반)
팀 이 름	결속밴드
지도교사	김도훈 선생님
연 구 자	모전중학교 김시훈

1. 탐구동기

학교생활에서 사진, 문서, 메시지 같은 디지털 자료를 친구나 선생님에게 전달할 일이 많다. 그러나 일반 파일은 전달 중 내용을 확인하거나 바꿀 수 있고, 파일의 크기가 크면 저장과 전송에도 불편이 생긴다. 이러한 문제를 해결하는 과정에 수학에서 배운 이산수학의 트리, 이진수, 부울대수 개념이 실제로 사용된다는 점이 흥미로웠다.

특히 자주 나타나는 데이터를 짧게 표현하는 허프만 코딩과 비트 단위 연산인 XOR의 성질을 직접 코드로 확인해 보고 싶었다. 단순한 암호 예시를 넘어 실제 웹 환경에서 사용할 수 있는 암호화 방식을 함께 적용하여, 메시지와 파일을 압축하고 안전하게 복원하는 Secure Message 웹앱을 만들었다.

2. 목적

첫째, 바이트의 출현 빈도와 이진 트리를 이용하는 허프만 압축의 원리를 이해하고 무손실 압축으로 구현한다. 둘째, XOR의 자기역원 성질을 학습용 비교 기능으로 구현한다. 셋째, 실제 보안에 적합한 PBKDF2와 AES-256-GCM을 적용하여 비밀번호가 없으면 내용을 읽을 수 없고 파일이 바뀌면 복호화를 거부하는 안전한 .enc 패키지를 만든다. 마지막으로 이산수학 개념이 데이터 압축, 암호화, 파일 구조 설계에 연결되는 과정을 시각화한다.

3. 탐구 내용 및 방법

1) 문제인식

디지털 자료를 전달할 때는 세 가지 조건이 필요하다. 첫째, 제3자가 내용을 읽지 못하도록 하는 기밀성이다. 둘째, 전달 중 파일이 바뀌었을 때 이를 알아내는 무결성이다. 셋째, 반복되는 데이터는 가능한 한 효율적으로 표현하는 것이다. 일반 텍스트나 파일을 그대로 저장하거나 공유하면 이러한 조건을 한 번에 확인하기 어렵다.

또한 단순 지판 암호나 반복 키 XOR 방식은 원리를 배우기에는 좋지만 실제 파일 보호에는 약점이 있다. 따라서 학습용 XOR과 실제 기본 방식인 AES-GCM을 분리하고, 사용자에게 두 방식의 차이를 보여 주는 프로그램이 필요하다고 판단하였다.

2) 가설설정

반복되는 문자가 많은 텍스트를 바이트 단위로 분석하여 허프만 코딩을 적용하면, 자주 나타나는 바이트에 짧은 이진 코드를 배정하므로 압축 전보다 데이터 크기가 줄어든 것이다. 또한 PBKDF2와 AES-256-GCM을 적용하면 같은 비밀번호를 사용하더라도 매번 다른 암호문이 만들어지고, 비밀번호가 틀리거나 암호문 및 헤더가 바뀐 경우에는 원본을 복원할 수 없을 것이다.

단, 이미 압축된 이미지·영상 파일이나 무작위에 가까운 데이터는 반복이 적어 허프만 압축률이 낮거나 패키지 정보 때문에 크기가 조금 커질 수 있다고 예상하였다.

3) 탐구방법

1. 텍스트 또는 25MB 이하 파일을 바이트 배열로 변환한다.
2. 0부터 255까지 각 바이트 값의 출현 횟수를 빈도표에 기록한다.
3. 최소 힙으로 빈도가 낮은 두 노드를 반복 결합하여 허프만 이진 트리를 만든다.
4. 접두어가 겹치지 않는 이진 코드로 데이터를 압축한다.
5. 비밀번호와 무작위 salt를 PBKDF2-SHA-256 250,000회에 적용해 AES-256 키를 만든다.
6. 무작위 IV와 AES-256-GCM으로 압축 데이터를 암호화하고 헤더도 인증한다.
7. 복호화 시 비밀번호 오류 또는 데이터 변조가 발생하면 인증 실패를 표시하고 중단한다.
8. 허프만 트리, 코드표, 압축 전후 크기, 바이트 분포 그래프를 화면에 표시한다.

[그림 1] Secure Message 처리 흐름

원본 입력 → 허프만 압축 → PBKDF2 키 유도 → AES-256-GCM 암호화 → SME2 .enc 패키지 → 인증 복호화 → 원본 복원

4. 이론적 배경과 구현

1) 바이트 빈도표와 허프만 이진 트리

컴퓨터 파일은 모두 0부터 255까지의 숫자인 바이트의 연속으로 저장된다. 프로그램은 입력 데이터에서 각 바이트가 몇 번 나타나는지 256칸 빈도표로 계산한다. 그다음 빈도가 가장 낮은 두 노드를 먼저 합치는 과정을 반복하여 허프만 이진 트리를 만든다. 자주 등장하는 바이트는 루트에 가까워져 짧은 코드가 되고, 드문 바이트는 긴 코드가 된다.

예를 들어 A가 5회, B가 2회, C가 1회 나타난다면 A에는 짧은 코드인 0, B와 C에는 10, 11과 같은 코드를 줄 수 있다. 어떤 코드도 다른 코드의 앞부분이 되지 않는 접두어 없는 코드이므로, 압축된 비트열을 왼쪽부터 읽어도 원래 바이트를 정확하게 구분할 수 있다. 이 방식은 원본을 완전히 되돌릴 수 있는 무손실 압축이다.

프로그램에서는 최소 힙을 사용하여 항상 가장 작은 빈도의 노드 두 개를 빠르게 선택한다. ZIP·PNG 등에 사용되는 DEFLATE도 허프만 코딩을 활용하는 대표적인 사례이다. 다만 이 프로젝트의 패키지는 복원 정보도 저장하므로, 짧은 데이터나 이미 압축된 파일에서는 전체 .enc 파일 크기가 줄지 않을 수 있다.

[그림 2] 허프만 트리와 접두어 없는 코드

웹앱의 분석 화면에서 바이트별 빈도, 이진 트리, 0·1 코드표를 확인한다.

2) XOR과 부울대수

XOR은 두 비트가 서로 다르면 1, 같으면 0이 되는 부울 연산이다. 같은 값을 두 번 XOR하면 원래 값으로 돌아오는 자기역원 성질이 있다. 즉, $A \text{ XOR } K \text{ XOR } K = A$ 이다.

이 성질을 이용하면 데이터 A에 키 K를 XOR하여 바꾸고, 같은 키 K를 다시 XOR하여 복원할 수 있다. 프로그램에는 이 원리를 직접 확인하는 학습용 XOR 비교 모드를 넣었다. 원본, 단순 반복 키 XOR 결과, SHA-256으로 확장한 반복 키 XOR 결과의 바이트 분포를 그래프로 비교할 수 있다.

그러나 반복 키 XOR은 같은 키의 반복 사용과 키 추측 공격에 취약하다. 따라서 이 방식은 부울대수 학습과 비교를 위한 기능일 뿐, 기본 파일 보호 방식으로 사용하지 않았다.

3) PBKDF2와 AES-256-GCM

실제 기본 암호화에는 웹 브라우저의 Web Crypto API를 사용하였다. 사용자가 입력한 비밀번호를 바로 AES 키로 쓰지 않고, 매번 새로 만드는 16바이트 salt와 함께 PBKDF2-SHA-256 250,000회를 적용하여 AES-256 키를 유도한다. 이 과정은 단순 비밀번호 추측을 어렵게 만드는 역할을 한다.

그 뒤 12바이트의 무작위 IV를 사용해 AES-256-GCM으로 압축 데이터를 암호화한다. AES-GCM은 내용을 숨기는 기밀성뿐 아니라, 암호문이나 중요한 헤더가 바뀌었는지도 확인한다. 따라서 비밀번호가 틀리거나 파일의 암호문·헤더가 변조되면 인증 검증에 실패하고 원본 복원을 거부한다. 같은 비밀번호와 같은 원본을 넣어도 salt와 IV가 매번 새로 생성되므로 암호문은 매번 달라진다.

4) SME2 .enc 패키지 설계

암호화 결과는 SME2라는 식별값을 갖는 .enc 패키지로 만든다. 패키지에는 파일 이름, MIME 형식, 원본 크기, 허프만 비트 길이, 256개 바이트의 빈도표, PBKDF2 반복 횟수, salt, IV, AES-GCM 암호문과 인증 태그가 들어간다. 이 중 헤더 전체를 AES-GCM의 추가 인증 데이터(AAD)로 사용하여 파일 이름이나 복원 정보만 바꾸어도 복호화를 거부하도록 하였다.

수신자는 .enc 파일과 같은 비밀번호를 입력하면 헤더를 읽고 AES-GCM 인증을 먼저 확인한 뒤, 허프만 트리로 압축 데이터를 풀어 원본 파일을 복원한다. 선택적으로 Supabase 보관함에 .enc 패키지만 올릴 수 있지만, 원본 파일과 비밀번호는 브라우저 밖으로 보내지 않도록 설계하였다.

[그림 3] SME2 .enc 패키지 구성

식별값·파일 정보·허프만 복원 정보·PBKDF2 정보·salt·IV·AES-GCM 암호문 및 인증 태그

5. 구현 결과 및 해석

Secure Message는 HTML, CSS, JavaScript로 만든 브라우저 기반 웹앱이다. 사용자는 텍스트 또는 파일과 비밀번호를 입력하고 암호화 버튼을 누른다. 프로그램은 바이트 빈도 계산, 허프만 트리 생성과 압축, PBKDF2 키 유도, AES-GCM 암호화, .enc 패키지 생성 순서로 처리한다. 복호화 화면에서는 같은 비밀번호를 입력해 인증 검증과 원본 복원을 확인할 수 있다.

구현 과정에서 허프만 트리와 각 간선의 0·1 코드, 바이트 분포 그래프, 압축 전후 크기를 화면으로 확인하도록 하였다. 반복이 많은 텍스트에서는 압축 비트열의 크기가 줄어드는 것을 확인할 수 있었고,

압축된 파일 또는 무작위 데이터에서는 압축 효과가 작을 수 있다는 점도 확인하였다. 이는 허프만 코딩의 효과가 데이터의 빈도 분포에 따라 달라짐을 보여 준다.

AES-GCM 모드에서는 잘못된 비밀번호를 입력하거나 .enc 파일의 내용 또는 헤더를 바꾼 경우 인증 검증에 실패하여 복호화가 중단되도록 구현하였다. 이 결과는 단순히 내용을 숨기는 것뿐 아니라 전달된 파일이 원래 상태인지 확인하는 무결성 검증도 필요함을 보여 준다.

코드 문법 검사는 `node --check script.js`로 확인하였고, Node Web Crypto 환경에서 AES-GCM 패키지를 암호화한 뒤 다시 복호화하는 왕복 테스트와 변조 거부 테스트를 수행하였다.

[그림 4] Secure Message 웹앱 실행 화면
암호화·복호화·분석·공유 보관함 화면에서 처리 결과를 확인한다.

6. 결론 및 한계

이번 연구를 통해 이산수학의 빈도표, 이진 트리, 최소 힙, 부울대수 XOR, 이진 데이터 표현이 실제 프로그램 기능과 연결됨을 확인하였다. 허프만 트리는 자주 나타나는 데이터를 짧은 코드로 표현하여 반복 데이터의 저장 효율을 높였고, XOR은 $A \text{ XOR } K \text{ XOR } K = A$ 라는 성질을 통해 비트 연산과 복호화 원리를 이해하는 데 도움을 주었다. 실제 보안 기능에는 PBKDF2와 AES-256-GCM을 사용하여 비밀번호가 틀리거나 데이터가 변경된 경우 복호화를 막았다.

다만 이 프로젝트는 교육용 브라우저 앱이다. 현재 Supabase 공유 보관함에는 로그인과 수신자별 권한 관리 기능이 없으며, 만료된 파일을 서버에서 자동으로 삭제하는 기능도 없다. 또한 25MB보다 큰 파일은 브라우저 메모리 사용을 고려하여 처리하지 않는다. 반복 XOR 모드는 비교용이므로 실제 파일 보호에 사용해서는 안 된다.

7. 소감

김시훈: 처음에는 암호화가 단순히 비밀번호를 넣어 파일을 잠그는 과정이라고 생각했다. 하지만 바이트 빈도, 이진 트리, 비트 연산, 키 유도, 인증 태그처럼 여러 수학과 정보 개념이 함께 사용된다는 것을 알게 되었다. 특히 허프만 트리를 화면으로 확인하면서 트리 구조가 실제 데이터 압축에 쓰이는 과정을 이해할 수 있었다. 또한 안전한 프로그램을 만들 때는 암호화뿐 아니라 파일이 바뀌지 않았는지 확인하는 기능도 중요하다는 점을 배웠다.

8. 향후 연구 계획

다음에는 서로 다른 종류의 파일을 대상으로 압축률, 패키지 크기, 처리 시간을 표로 기록하여 허프만 압축의 효과를 더 자세히 비교하고 싶다. 또한 실제 서비스에 가깝게 발전시키기 위해 로그인, 수신

자별 접근 권한, 업로드 횟수 제한, 악성 파일 검사, 만료 파일 자동 삭제를 추가하는 방법을 탐구하고 싶다. 사용자가 빈도표와 키 길이를 바꾸어 결과를 비교할 수 있는 실험 모드도 추가하고자 한다.

9. 참고문헌

- David A. Huffman, A Method for the Construction of Minimum-Redundancy Codes, Proceedings of the IRE, 1952.
- NIST, Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM) and GMAC, SP 800-38D, 2007.
- NIST, Recommendation for Password-Based Key Derivation: Part 1: Storage Applications, SP 800-132, 2010.
- MDN Web Docs, Web Crypto API, https://developer.mozilla.org/en-US/docs/Web/API/Web_Crypto_API (접속일: 2026. 7. 29.).
- IETF, RFC 8018: PKCS #5: Password-Based Cryptography Specification Version 2.1, 2017.